

```

public class BinarySearchTree {

    TreeNode root;

    public void insert(char letter) {
        TreeNode newNode = new TreeNode(letter);
        this.root = insertTreeNode(this.root, newNode);
    }

    private TreeNode insertTreeNode(TreeNode root, TreeNode newNode) {

        if (root == null) {
            root = newNode;
            return root;
        }

        if (newNode.letter <= root.letter) {
            root.left = insertTreeNode(root.left, newNode);
        } else if (newNode.letter > root.letter) {
            root.right = insertTreeNode(root.right, newNode);
        }

        return root;
    }

    public void inorder() {
        inorderTraversal(this.root);
    }

    private void inorderTraversal(TreeNode root) {
        if (root != null) {
            inorderTraversal(root.left);
            System.out.print(root.letter + " ");
            inorderTraversal(root.right);
        }
    }

    public void preorder() {
        preorderTraversal(this.root);
    }

    private void preorderTraversal(TreeNode root) {
        if (root != null) {
            System.out.print(root.letter + " ");
            preorderTraversal(root.left);
            preorderTraversal(root.right);
        }
    }
}

```

```

    }

}

public void postorder() {

    postorderTraversal(this.root);
}

private void postorderTraversal(TreeNode root) {
    if (root != null) {

        postorderTraversal(root.left);
        postorderTraversal(root.right);
        System.out.print(root.letter + " ");
    }

}

public String deapthFirstSearch(char value) {
    Stack stack = new Stack();
    stack.push(root);
    while (!stack.isEmpty()) {

        TreeNode temp = stack.pop();
        System.out.print("<->" + temp.letter);
        if (temp.letter == value) {
            return "found";
        } else {
            if (temp.right != null) {
                stack.push(temp.right);
            }
            if (temp.left != null) {
                stack.push(temp.left);
            }
        }
    }

    return "not found";
}

public String BFS(char value) {
    int level = 0;
    if (this.root == null) {
        return "tree is empty";
    } else {
        Queue queue = new Queue();

```

```

queue.enqueue(root);

while (!queue.isEmpty()) {

    TreeNode temp = queue.dequeue();
    System.out.print("->" + temp.letter);
    if (temp.letter == value) {
        return "level =" + level;
    } else {

        if (temp.left != null) {
            queue.enqueue(temp.left);
            level++;
        }
        if (temp.right != null) {
            queue.enqueue(temp.right);
            level++;
        }
    }
    ;
}

return "not found";

}
}

public int getLevel(char letter) {
    int depth = TreeLevel(root, letter, 0);
    return depth;
}

private int TreeLevel(TreeNode root, char letter, int level) {

    if (root == null) {
        return 0;
    }
    if (root.letter == letter) {
        return level;
    }
    int result = TreeLevel(root.left, letter, level + 1);
    if (result != 0) {
        // If found in left subtree , return
        return result;
    }
    result = TreeLevel(root.right, letter, level + 1);

    return result;
}

```

```

    }

}

public class TreeNode {

    char letter;
    TreeNode right;
    TreeNode left;

    public TreeNode(char data) {
        this.letter = data;
        this.right = null;
        this.left = null;
    }

}

```

```

public class QNode {

    TreeNode node;
    QNode next;

    public QNode(TreeNode node) {
        this.node = node;
        this.next = null;
    }

}

public class Queue {

    QNode front;
    QNode rear;

    public void enqueue(TreeNode node) {

        QNode new_node = new QNode(node);
        if (this.front == null) {
            this.front = new_node;
            this.rear = new_node;
        } else {

            rear.next = new_node;
            this.rear = new_node;
        }
    }

    public TreeNode dequeue() {

```

```

        if (this.front == null) {
            return null;
        }
        QNode temp = this.front;
        this.front = this.front.next;

        if (this.front == null) {
            this.rear = null;
        }
        return temp.node;
    }

    public boolean isEmpty() {
        if (this.front == null) {
            return true;
        }
        return false;
    }
}

```

```

public class StackNode {

    TreeNode node;
    StackNode next;

    public StackNode(TreeNode node) {
        this.node = node;
        this.next = null;
    }
}

public class Stack {

    StackNode top;

    public void push(TreeNode data) {
        StackNode new_node = new StackNode(data);

        new_node.next = top;
        this.top = new_node;
    }

    public TreeNode pop() {

```

```

        if (this.top != null) {
            StackNode temp = this.top;
            this.top = top.next;
            return temp.node;
        }
        return null;
    }

    public boolean isEmpty() {
        if (this.top == null) {
            return true;
        }
        return false;
    }
}

```

```

public class Driver {

    public static void main(String[] args) {
        BinarySearchTree BST = new BinarySearchTree();

        BST.insert('N');
        BST.insert('O');
        BST.insert('N');
        BST.insert('L');
        BST.insert('I');
        BST.insert('N');
        BST.insert('E');
        BST.insert('A');
        BST.insert('R');
        BST.insert('D');
        BST.insert('A');
        BST.insert('T');
        BST.insert('A');
        BST.insert('T');
        BST.insert('Y');
        BST.insert('P');
        BST.insert('E');

        System.out.println(BST.getLevel('E'));

        BST.deapthFirstSearch('T');
        System.out.println("");
        BST.BFS('T');
    }
}

```

